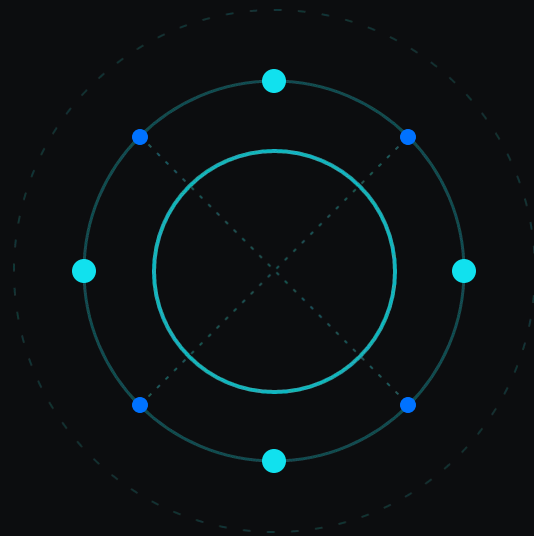


# Airflow Autopilot

Stefan Wang

Sandeep Dhillon

Data Infrastructure @ LinkedIn



# Letting engineers focus on product intent, not pipeline machinery

## 01

### The Problem

Writing pipeline code is no longer the main bottleneck. The challenge is navigating infrastructure complexities:

- Finding trustworthy data
- Validating schema joins
- Testing without touching prod

## 02

### The Agentic Layer

Airflow Autopilot acts as the intelligent bridge, translating raw user intent into complete operational workflows:

- Autonomous discovery
- Safe, isolated sandbox runs
- Comprehensive validations

## 03

### The Outcome

Airflow remains the orchestration engine, but its operational surface becomes completely transparent:

- Checked, verified data products
- Ready for production review
- Zero infrastructure overhead

# Airflow Scale & Use Cases

**100K+**

**Workflows / Day**

Platform and user data pipelines

**50K+**

**DAGs Defined**

Unique number of DAGs registered

**65%+**

**Growth**

Year over year increase

## Key Production Use Cases

### Product & Data Pipelines

Powering LinkedIn's core product workflows. Running large-scale offline compute, Spark, and Flink jobs to process large-scale datasets.

### Data Triggers & Streams

Orchestrating real-time data execution and system-wide backfills. Managing seamless data pushes to high-performance stores like Pinot, Kafka, Ambry, and Venice.

# Writing code is the easy part

**01**

## Intent

What should the data do?

**02**

## Discovery

Which source can we trust?

**03**

## Execution

Will it run here?

**04**

## Validation

Is the output right?

# Start with the intent / desired outcome

| or run unreviewed third-party plugins/hooks/scripts from the internet. See [go/ai-dev-safely](#) for more info.

/clear

Build a daily Airflow pipeline on the holdem grid that produces a per-campaign daily aggregate of ad performance, written as one daily-partitioned table keyed by campaign. Process the day at logical\_date - 1, America/Los\_Angeles, and support backfills.

Aggregate from raw ad-engagement tracking events, not from an existing daily ads-analytics fact. Use the full engagement set - impressions, clicks, video actions, and leads - each in both onsite and offsite/RTB form. Also enrich each campaign with its latest pacing and delivery-control state.

Per campaign per day, emit counts for each engagement type, spend, and derived CTR/VTR.

I don't know the exact source tables - discover and verify them yourself, and pick a sensible output table name in my namespace.

I

\* Tomfoolering... (5s · ↓ 25 tokens · thought for 4s)

└ Tip: Set an objective with /goal - Claude keeps working until it's met

> █

# Find the right data, then check it

Read 2 files

This is the ideal template. `etl_with_datasensor` gives me the exact pattern: `DatasetSensorArray + PartitionSensorDefinition` for the gate, `SparkBatchOperator` with `@Artifact(starter_spark_scala_2.12, azkabanHoldem)` for the compute, the `LA-tz partition_date` macro, and `app_params=[input, output, partition_date]`. Let me create the task list and read the remaining references – the sensors (for daily/hourly backfill switching) and the Scala job scaffold.

Read 2 files

Now I have all the conventions:

- **Daily** sensor: `granularity='DAILY'`, value `YYYY-MM-DD-00`, dataset `tracking.<Event>`
- **Hourly** sensor: `granularity='HOURLY'`, value `YYYY-MM-DD-HH`, dataset `tracking_hourly.<event>`

That maps cleanly onto the backfill clause (`live→hourly@-23`, `backfill→daily@-00`). Now the mandatory gate: verify each signal resolves to a `gridTable` in **both** daily and hourly form with a writer. Let me batch-resolve the canonical names.

Called captain 8 times

All four RTB + lead signals confirmed: `gridTable` in **both** daily (`tracking.*`) and hourly (`tracking_hourly.*`) form on holdem. Now the onsite forms and the enrichment sources.

Called captain 8 times

Most signals confirmed `gridTable` in both forms with lineage on holdem. Two need a writer check: the **onsite impression** (`GenericImpressionEvent_sponsored`) `gridTable` forms show `has_lineage: false`, which is a yellow flag. Let me confirm a writer exists for both its forms before accepting it.

Calling captain 3 times...

# Ask only when the answer matters

|       |               |                                               |
|-------|---------------|-----------------------------------------------|
| Leads | adleadevent ✓ | unverified – no clear RTB lead event surfaced |
|-------|---------------|-----------------------------------------------|

Each event family exists in both a `tracking_hourly.*` form and a daily form (→ drives the backfill direction clause). Pacing/delivery-control enrichment grounds to `u_metrics.bidding_pacing_union` (pacing) + `tracking.AdDeliveryGoalChangeEvent` (delivery control). Measures (counts, spend, CTR, VTR) are canonical (`lms_*.ctr`, `*.vtr`, `*.clicks`, etc.).

Two things genuinely change the build and discovery can't settle them confidently, so I'll confirm those – everything else I'm defaulting from convention.

## Proposed build-goal:

Inputs (raw tracking events, onsite + offsite/RTB each):

impressions · clicks · video actions · leads

Enrichment: latest per-campaign pacing + delivery-control state

Grain / key: one row per campaign per day

Measures: per-type counts, spend, derived CTR + VTR

Output: daily-partitioned table keyed by campaign (net-new, your namespace)

Window: daily, day = `logical_date - 1`, America/Los\_Angeles

Backfill: live → hourly source @ hour -23; backfill → daily source @ hour -00

Flagged: leads RTB side unverified (drop if no gridTable form)

pacing source defaulted to `bidding_pacing_union` + `AdDeliveryGoalChangeEvent`

← ☐ Leads coverage ☐ Output table ✓ Submit →

> 1. Onsite leads only

Use `adleadevent` (onsite) only. The other 3 types stay onsite/offsite/RTB. Matches what discovery can actually verify: build drops the RTB-lead side.

# Build the whole pipeline

All 7 engagement signals verified as gridTable in both daily + hourly form on holdem. Output: per-type counts (onsite/offsite split), spend, CTR, VTR, plus the pacing/delivery-control columns.

**Backfill (the HARD requirement) – implemented, not deferred**

Live vs backfill is decided at task-execution time off `dag_run.run_type`. Since `PartitionSensorDefinition` doesn't template `partition_check_granularity`, I added a 3-line `BackfillAwarePartitionSensorDefinition` subclass so granularity flips too:

- Live → hourly form, gate on last hour -23, Spark reads all 24 hourly partitions.
- Backfill → daily form, gate on -00, Spark reads the single daily partition. Never inverted.

**Validation status**

-  Scala compiles (`gradlew :starter_spark_scala:compileScala`, exit 0)
-  DAG parses + `passes picli airflow enforce-policy` ("All DAGs pass policy checks")
-  Live rdev run (real partition gating, backfill switching, one-partition write) – not yet run

The remaining item is a live end-to-end on an rdev, which is a long, outward-facing operation (rdev spin-up, build/release, DAG upload, trigger, observe), possibly with auth/MFA prompts – so I'm checking before kicking it off.

User answered Claude's questions:

- └ · Static validation is green (Scala compiles, DAG parses + passes policy). Do you want me to run the full live rdev end-to-end now? → Review the code with me first

Let's walk through the two files. I'll focus on the parts where the logic actually matters and where you'd want to second-guess me.

**The DAG – how live vs backfill flips**

Everything keys off one render-time switch:

# Test in a real Airflow sandbox

- Background command "Poll GGW connection endpoint until grid setup completes" completed (exit code 0)

The connection poller fired with exit 0 – the GGW connection seeded. The SSO went through quickly. Let me confirm and check the grid setup completed cleanly.

Ran 1 shell command

Grid setup **completed successfully** (exit 0) – added grid + arms connections, made me an Airflow admin, and restarted Airflow. The 000 was just the mid-restart blip. Let me re-verify the restarted.

Ran 1 shell command

**Preflight passes** (post-restart): scheduler + metadatabase healthy, GGW connection 200, DAG unpaused, 0 import errors. Task #2 done.

## SANDBOX READY

- ✓ SSO session active
- ✓ Compute gateway connected
- ✓ Scheduler and metadata database healthy
- ✓ DAG loaded, 0 import errors
- ✓ Output isolated from production

# Stay with the run

```
airflow_starter_kit — Test scheduled_campaign_daily_aggregate DAG on dev e2e — caffeinate · claude --permission-mode auto --effort max --plugin-dir ~/workplace/oklahoma-managed-airflow/llm-productivity-agents/linkedin-plugins/specialized/airflow-autopilot — 225x59
```

Monitor is live and will report each transition. Quick status while it runs:

Run `manual_e2e_proc20260621` is progressing well. The hardest-to-get-right part — the live/backfill branch + the per-form sensor wiring (the thing fixed in #271) — is **working**: branch chose hourly, daily got skipped, hourly sensor is poking the right partitions.

Remaining: hourly sensor clears → Spark aggregate (the heavy 800-executor job) → DQ validation. I'll report as each task transitions. The sensor pokes every 5 min, so if the `2026-06-21-23` partitions are all present it clears on the first poke; the Spark stage is the long pole.

\* Churned for 10m 21s · 1 monitor still running

Monitor event: "campaign DAG run task-state transitions"

Sensor **cleared on the first poke** — all nine `2026-06-21-23` hourly partitions are present. The Spark aggregate (`aggregate_campaign_daily`) is now **running**. That's the long pole.

Let me grab the Spark application/tracking URL from the task log so I can watch the job's actual progress (and confirm it's not just stuck queuing for 800 executors).

Ran 1 shell command

Spark submission is **completely clean** — the app is RUNNING on holdem. The critical validation here is that the DAG→Spark contract rendered exactly right:

params: tracking\_hourly.adimpressionsevent ... (9 hourly tables)  
`2026-06-21 HOURLY u_svc_pr_oklahoma.scheduled_campaign_daily_aggregate 60`

All 13 app\_params, correct order: 9 hourly input tables → `dateKey=2026-06-21` → `HOURLY` → output table → retention `60`. ✓ Artifact resolved to `azkabanHoldem.zip 0.0.214`, executing as `svc_pr_oklahoma`, `production.marker=false` (sandboxed test).

- **Spark app**: `application_1782158476827_279762` (provisioned 800 executors quickly — no queue starvation)
- **Spark UI**: `http://ltx1-titanwp01.grid.linkedin.com:8080/proxy/application_1782158476827_279762/`

Now it's just the job crunching a full day of hourly ad-engagement data. The monitor will report when `aggregate_campaign_daily` finishes → then DQ validation. Waiting.

\* Crunched for 49s · 1 monitor still running

```
check the spark job log is it progressing
└─ 24 agent types available
└─ 32 skills available
```

# Fix the problem, then run it again

1

## DIAGNOSE

input\_form=hourly confirmed ✓. Waiting for the sensor poll result.

Read shell output

Thought for 16s

check\_campaign\_inputs\_hourly → **success** (sensor cleared first poke — hourly data landed). But aggregate\_campaign\_daily went to **up\_for\_retry** within seconds of starting — that's a *submission-time* failure (a real 16-min Spark job can't fail that fast), not a compute failure. Let me load the logs skill and pull attempt 1's log to see why.

Loaded skill logs

2

## RERUN

Thought for 8s

aggregate\_campaign\_daily = **SUCCESS** at 07:32:58Z — the Spark aggregate completed cleanly (~49 min under load). Now validate\_campaign\_daily (the DQ task) is running. Baseline DQ was ~5.5 min. Waiting for the DAG run to finish.

Read shell output

Thought for 16s

 **DAG RUN TERMINAL: SUCCESS** at 07:38:06Z. Full pipeline passed end-to-end: branch → sensor → Spark aggregate (success) → DQ validate (success). Let me gather the pipeline insights (output table/partition, DQ violations and any task exceptions) for the final report.

# Finish with checked data

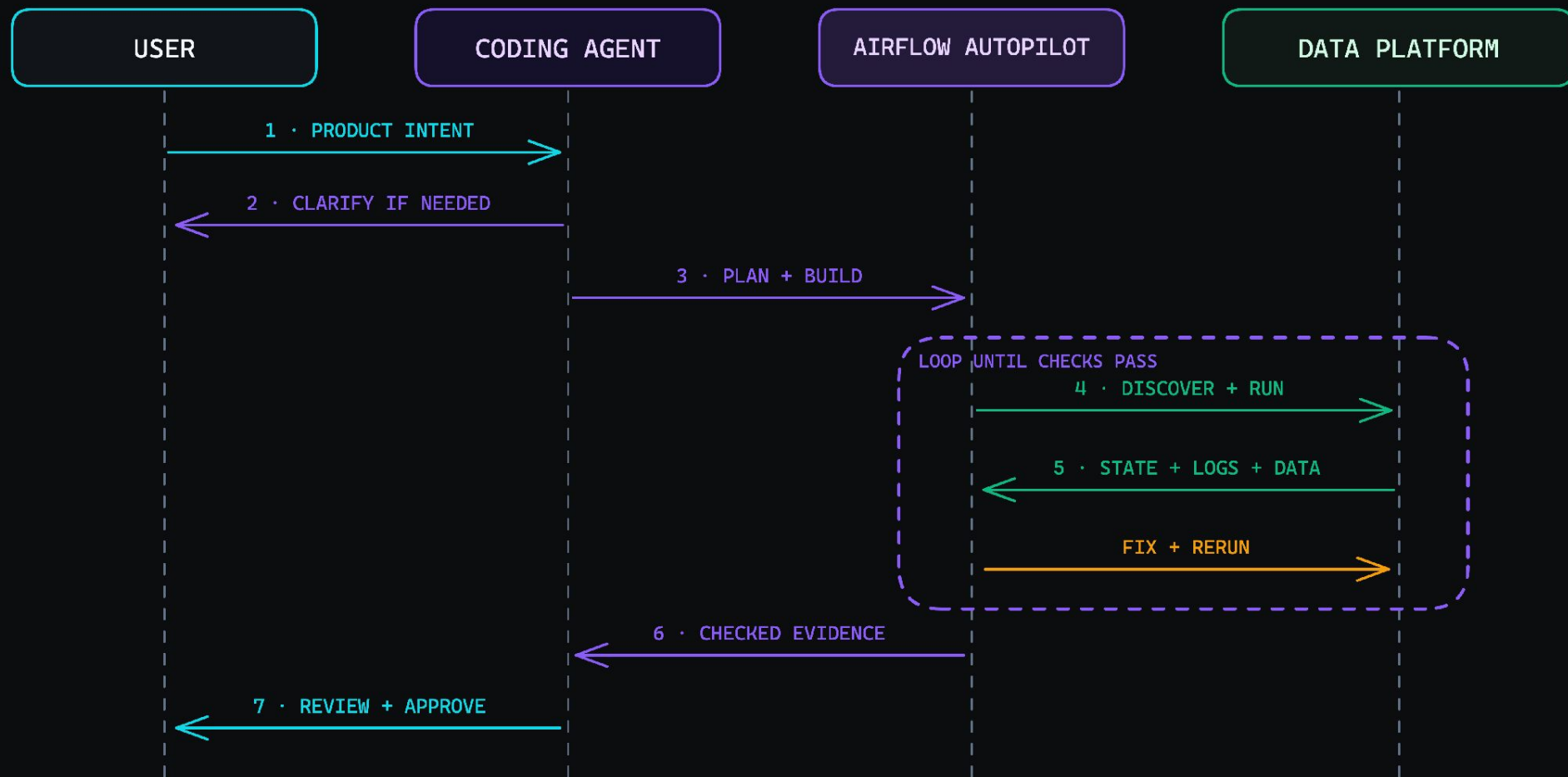
|                                               |                                                                                       |
|-----------------------------------------------|---------------------------------------------------------------------------------------|
| <code>choose_input_form</code>                | ✅ success – picked <b>hourly</b> (live path)                                          |
| <code>check_campaign_inputs_daily</code>      | ✅ skipped (correct – branch chose hourly)                                             |
| <code>check_campaign_inputs_hourly</code>     | ✅ success – cleared on 1st poke (all 9 <code>2026-06-21-23</code> partitions present) |
| <code>aggregate_campaign_daily</code> (Spark) | ✅ success – ~16 min, 786 executors, 20k+ tasks, <b>0 failures</b>                     |
| <code>validate_campaign_daily</code> (DQ)     | ✅ success – DSSv3 <b>SUCCEEDED</b> , all hard assertions passed                       |

\* Worked for 3m 24s

## any data insights?

- Good question – the run wrote a real partition, so let's pull actual numbers out of it. The richest insight is `2026-06-21-00`). Let me load the query tools and also re-pull the now-complete Spark driver log (the job does
- `execute_trino_query` (read-only, `[internal target]`) is the right tool. Let me run an aggregate summary of

# THE USER STATES INTENT. THE AGENT CARRIES THE LOOP.



Airflow Autopilot

# Architecture

Going behind the scenes

---

# Agentic Loop & Integration

## USER ENVIRONMENT

### Github Copilot As Agent Harness

Acts as the primary AI agent harness running locally on the **user's laptop** to coordinate reasoning, code generation, and environment operations.

#### AIRFLOW AUTOPILOT :: COPILOT PLUGIN

Injects tailored system instructions, rules, and **best practices for building robust pipelines**. Collection of **several skills**.

#### GIT BRANCH ISOLATION

Pushes generated DAG code safely to **separate test branches**.



## MCP

### Data Discovery

Exposes schema, catalogs, and lineage records directly to the harness context.

### Context Search

Unified semantic querying across systems, documentation, and history.

### CLI Tools for Containerized Airflow

MCP integration enabling the local agent to programmatically initialize, execute, and inspect **Airflow in a remote container environment** dynamically.

# Airflow on Remote Container

## INFRASTRUCTURE

### Kubernetes Native Deployment

Engineered to run natively on **K8s** for seamless orchestration, scalability, and robust resource isolation.

## CONNECTIVITY

### Source Control & Compute Access

Maintains secure, integrated access to **internal git repositories** and core **compute clusters** with native capabilities to submit Spark applications.

## DEVELOPMENT

### Complete Remote Environment

Fully loaded with common **company internal tools** to function as a self-contained, turnkey developer workspace.

## AUTOPILOT INTEGRATION

### Isolated Execution Space

Provides a completely partitioned Airflow environment specifically designed to run and validate **Autopilot generated DAGs** safely.

# Data Discovery: Find the Right Tables

Turn a massive table catalog into an understandable knowledge system. Match business intent to the right data.

## 1. RAW CATALOG

Thousands of Company  
Tables

fact\_sales\_daily

dim\_customer\_v2

user\_session\_logs

prod\_orders\_final

⋮

Metadata lacks business  
context and usability signals

## 2. UNIFIED CONTEXT LAYER

### DB Catalog Metadata

Structured attributes mapped directly from source data: **Names, schemas, descriptions, ownership, and upstream lineage.**

### OPS Operational Signals

Real-time telemetry and metadata analysis: **Usage patterns, frequency stats, execution history, and active user counts.**

### AI AI-Enriched Summaries

Automated semantic understanding from an **offline AI pipeline** that synthesizes and labels intent-driven metrics.

### EDU Curated Knowledge

**Human-curated** global and team insights: **definitions, critical caveats, best practices, and verified sample queries.**

**Result: Each dataset gets a dynamic, highly searchable context unit.**

## 3 & 4. RETRIEVAL & SHORTLIST

NATURAL LANGUAGE INTENT  
"Active enterprise customer ARR..."

**Hybrid Keyword + Vector**  
Semantic & Keyword Retrieval

**Intelligent Reranking**  
Context & Relevance Scored

### RANKED TABLE SHORTLIST

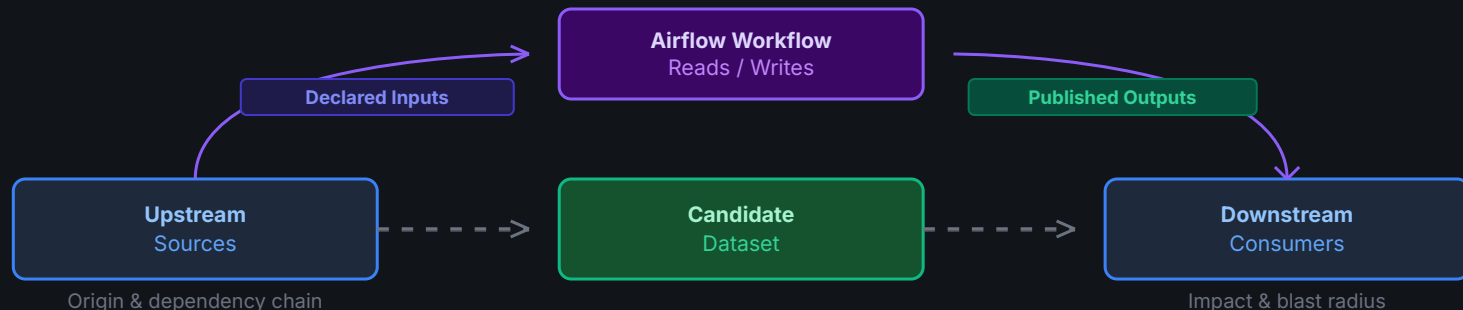
1 finance.ent\_customer\_arr

2 mktg.mrr\_waterfall\_v4

3 sales.account\_plans

# Data Lineage in Airflow Autopilot

## Interactive Dependency Map



## How Lineage Improves Data Selection

When search returns multiple similarly named datasets, lineage identifies each candidate's **origin**, **dependencies**, **consumers**, and role in the data flow.

This helps you select based on semantic meaning and dependency path, avoiding risky upstream structures.

# Unified Context Search

## 1. QUERY ANALYSIS

Analyze user query

Formulate searches for relevant sources.

Instead of basic keyword lookup, it decodes engineering intent to target the right specialized platforms.

## 2. FEDERATED FAN-OUT

The engine fans out across multiple distributed sources:

- **System Documentation**
- **Code search**
- **Wikis**
- **Slack**
- **Jira**

## 3. DEEP ENRICHMENT

For top results, instead of relying on shallow snippets:

- Fetch full **code files**
- Complete **Wiki / documentation content**
- 

Provide deep semantic context for resolution.

## 4. MERGE & RANK

Everything is normalized, ranked, and deduplicated into a single unified result set with:

- **relevance scores**
- **source metadata**
- **URLs**
- **references** for deeper reading

# The Airflow Autopilot Agentic Loop

## Putting It All Together: End-to-End Orchestration Loop

### 1. USER INTENT

#### Context & Rules

User defines key pipeline goals, required schemas, and interactive validation guardrails.

### 2. DISCOVERY

#### Data Discovery

Federated semantic search decodes pipeline intent, scanning across tables and lineage maps.

### 3. CODE GEN

#### Plan & Build

AI-native code generation structures logic, automatically constructing Python DAG, Spark apps code.

### 4. TEST & FIX

#### Self-Correct

Isolated trial runs execute programmatically, self-healing code until fully verified.

### 5. FINALIZE

#### Review & PR

Synthesizes verification logs, auto-generating clean pull requests for human validation.

### GITHUB COPILOT

#### Agent Harness

Provides access to frontier models and coordinates directly with secure environments via the agent harness.

### AGENT TOOLS

#### MCP

MCP provides access to company internal tools for unified context search, data discovery, lineage and launching remote Airflow Clusters.

### AUTOPILOT SKILLS

#### Agent Execution Logic

Translates declarative intent and rule parameters into physical action sequences.

### REMOTE AIRFLOW CLUSTER

#### Isolated Target Platform

Secure remote environment to provision, test, and programmatically verify generated DAGs.

# Next Evolution



## TODAY

### Airflow Autopilot

- Discover data
- Build pipeline
- Test pipeline
- Validate



## NEXT

### AI-native data platform

- **Everything in Autopilot**
- Prove correctness
- Promote to production
- Auto-diagnose in production
- Self-Heal pipelines

# Thank you • Q&A



Stefan Wang

Scan to Connect

Share Feedback

Sandeep Dhillon



Data Infrastructure @ LinkedIn

